



İST266 HİPOTEZ TESTLERİ NOKTA TAHMİNİ ÖDEVLERİ

Hazırlayan:

2220329066- Duru Topaloğlu

Ders Sorumluları:

Doç. Dr. Yasemin Kayhan Atılğan

Arş. Gör. Dolunay Ezgi Seyhan

Ödev 1

Kitle Parametrelerinin Hesaplanması:

μ (mu): kitle ortalaması

N: toplam gözlem sayısı

x_i : i. gözlem değeri olmak üzere,

Kitle Ortalaması = $49.856 \approx 50$

Kitle Varyansı = $100.043 \approx 100$ olarak hesaplanmıştır.

Farklı Örneklem Büyüklükleri ile Nokta Tahmini

Oluşturulan kitleden $n = 10$, $n = 30$, $n = 100$, $n = 500$ büyüklüklerinde örneklemeler oluşturulup, bu örneklemeler üzerinden ortalama ve varyans hesaplanarak nokta tahmini araçları elde edilmiştir.

Örneklem Boyutu (n)	Örneklem Ortalaması	Örneklem Varyansı	Ortalama Farkı	Varyans Farkı
n = 10	52.433	62.665	2.576	-37.378
n = 30	50.224	53.885	0.368	-46.158
n = 100	50.355	86.420	0.498	-13.623
n = 500	50.069	112.224	0.212	12.180

Örneklem büyüdükçe yapılan nokta tahminlerinin, bölüm 1’de hesaplanan kitle parametre değerlerine yaklaştığı gözlemlenmektedir.

Yansızlık ve Tutarlılık Simülasyonu

$n = 10$, $n = 30$, $n = 100$, $n = 500$ büyüklüklerindeki örneklem için yansızlık ve tutarsızlık özellikleri incelenecektir.

Kitleden çekilen her n değeri için 1000 adet örneklem ortalaması ve varyans aşağıdaki gibi elde edilmiştir:

$n=10$ Ortalama: 50.098 Varyans: 10.246

$n=30$ Ortalama: 49.929 Varyans: 3.220

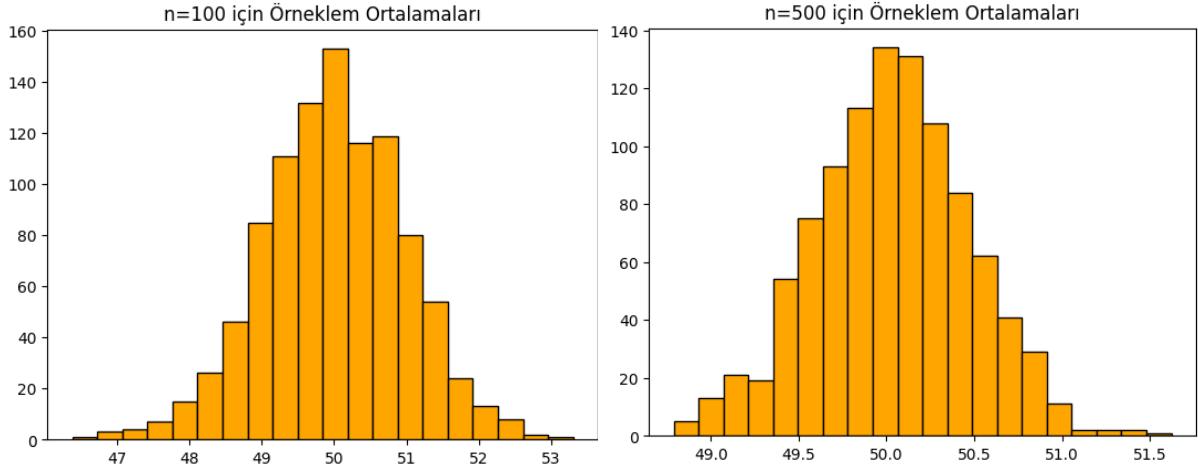
$n=100$ Ortalama: 50.000 Varyans: 0.958

$n=500$ Ortalama: 50.031 Varyans: 0.192

Örneklem büyüklüğü arttıkça tahmin edilen parametreye bir tahmin edici ne kadar yaklaşma eğiliminde ise, bu tahmin edici o kadar tutarlıdır. Değerlere bakıldığında n büyüdükçe (örneklem büyüdükçe) kitle ortalaması olan 50 değerine yakınlaşmakta ve varyans değerleri de gittikçe küçülerek 0 değerine yaklaşmaktadır.

İncelenen n değerleri için tahmin edinin beklenen değeri (ortalaması) kitle ortalamasına eşit ise yansızlık sağlanmış olur. Değerler yine kitle ortalaması olan 50'ye yaklaştığı için yansız olduğu gözlemlenmektedir.

Örneklem büyüklüğü arttıkça tahminlerin parametre etrafında yoğunlaştığı aşağıda verilmiş olan $n=100$ ve $n=500$ histogram grafiklerine bakılarak yorumlanabilir:



Örneklem büyüklüğü arttıkça tahminlerin doğruluğu ve güvenilirliği artar. Bu nedenle büyük bir örneklemle çalışmak gerçek kitle değerine en yakın sonuca ulaştırır.

Ödev 2

Simülasyon

Bu bölümde T1, T2, T3 gibi tahmin edicilerin kitleyi tahmin etme performanslarının karşılaştırılması için uygun hesaplamalar yapılmıştır.

Tahmin Edicilerin Performansı

Kitle ortalaması için:

Tahmin Edici	Yansızlık	Varyans	HKO
T1	19.7835	7.6414	7.6882
T2	20.0463	114.0191	114.0213
T3	6.6685	87.7064	265.4356

T2'nin T1'e göre etkinliği: 0.0670

Kitle varyansı için:

Tahmin Edici	Yansızlık	Varyans	HKO
T1	230.7791	3674.1001	3707.4978
T2	232.9397	116238.0624	116301.1018

T2'nin T1'e göre etkinliği: 0.0316

Kitle ortalaması ve kitle varyansı tahmin edicilerinin etkinlik hesaplaması için her iki tablodan da T1 ve T2 olarak seçilmiştir.

Her iki parametre için de T1 tahmin edicisi en iyi performansı göstermiştir. Her iki tabloda da T1 kitle parametrelerine en yakın değerlere sahiptir. (Yansızlık ve düşük varyanslılık sağlanmıştır.)

HKO (Hata Kareler Ortalaması) matematiksel formülünden de yola çıkılacağı üzere hem yansızlık hem de varyans büyüklüğünü hesaplayarak daha iyi bir tahmin edici sonucunu verir. Karşılaştırmalarda bu yüzden de en düşük HKO değeri değerlendirmeye alınır.

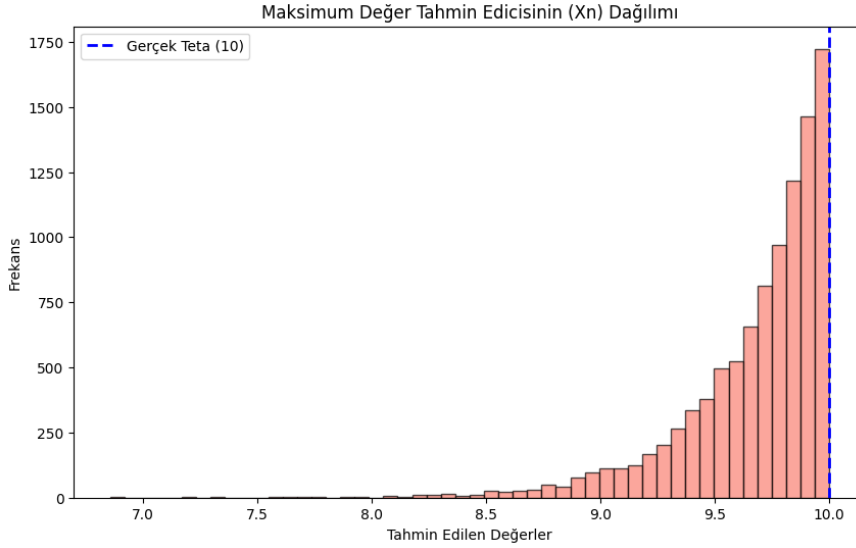
Ödev 3

$U(0,10)$ dağılımından $n = 30$ büyüklüğünde örneklem üretilmiştir. Her örneklem için: $\theta = X_n$ hesaplanmış ve işlem 10000 kez tekrar edilmiştir.

Tahmin Edicinin Davranışı

Tahminlerin Ortalaması: 9.677

Tahminlerin Varyansı: 0.099



Uniform dağılımında tüm değerlerin theta değerinin altında kalması gerekir. Dolayısıyla en büyük sıralı istatistik de bu dağılım kapsamında maksimum değeri üreteceği için, gerçek üst sınır olan theta'ya en yakın değeri vereceğinden dolayı iyi bir tahmin edici olarak kabul edilebilir. Ayrıca 10000 simülasyon sonucunda ortalamanın 9.67 çıkması tahmin edicinin yanlı olduğunu gösterir.

Örneklem büyüklüğü arttıkça histogram grafiğinden de görülebileceği üzere tahmin değerleri kitle değeri olan $\theta=10$ 'a yaklaşmaktadır. Bu durum ödev 1'de de açıklandığı üzere tutarlılık özelliği ile açıklanır.

Kodlar

```
[1]
0
✓ sn.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Ödevlerde kullanılan kodlar için gerekli olan kütüphaneler çağırılmıştır.

ÖDEV 1

```
[2]
0
✓ sn.
mu = 50
sigma = 10
N = 10000
```

```
[3]
0
✓ sn.
kitle = np.random.normal(mu, sigma, N)
np.random.seed(126)
```

Ödev 1 yönergesinde verilen dağılım bilgileri tanımlanarak, N=10000 olacak şekilde kitle oluşturuldu. random.seed fonksiyonu kodun her çalıştırıldığında aynı sonucu vermesi için kullanılır.

```
[4]
0
✓ sn.
kitle_ortalami = np.mean(kitle)
kitle_varyansi = np.var(kitle)
```

Kitlenin ortalaması ve varyansı hesaplanır.

```
[5]
0
✓ sn.
print(kitle_ortalami)
print(kitle_varyansi)

50.16023072653938
97.15035382862288
```

```
[6]
0
✓ sn.
np.random.seed(126)
n_degerleri = [10, 30, 100, 500]
for n in n_degerleri:
    orneklem = np.random.choice(kitle, size=n, replace=False)
    orneklem_ortalami = np.mean(orneklem)
    orneklem_varyansi = np.var(orneklem)

    ort_fark = orneklem_ortalami - kitle_ortalami
    var_fark = orneklem_varyansi - kitle_varyansi

    print(orneklem_ortalami)
    print(orneklem_varyansi)
    print(ort_fark)
    print(var_fark)
```

```
... 53.23864560686293
68.69051675716752
3.0784148803235496
-28.459837071455368
49.26395424972931
96.24849450585926
-0.8962764768100726
-0.9018593227636273
50.8172969879749
74.7518822625303
0.6570662614355172
-22.39847156609258
50.98302166626175
103.22374781097598
0.8227909397223669
6.073393982353096
```

n=10 , n=30, n=100 ve n=500 büyüklüğünde örneklem oluşturulur. Daha sonra ayrı ayrı bu örneklemelerin ortalamayı ve varyansı hesaplanır. Not: Örneklem seçimi np.random.choice fonksiyonuyla elde edilmiştir.

```
[7] 5 sn.
▶ n_degerleri = [10, 30, 100, 500]
tekrar_sayisi = 1000
simulasyon_ortalamlari = {}
```

```
[8] 0 sn.
▶ for n in n_degerleri:
    ortalamlar_listesi = []

    for i in range(tekrar_sayisi):
        # 1. Kitleden n gözlem içeren rastgele örneklem
        orneklem = np.random.choice(kitle, size=n, replace=False)

        # 2. Örneklem ortalaması
        ort = np.mean(orneklem)

        # 3. Listeye ekleme
        ortalamlar_listesi.append(ort)

    # Her n değeri için 1000 adet ortalamayı kaydet
    simulasyon_ortalamlari[n] = ortalamlar_listesi
```

Hesaplamalar için kullanılacak ortalamlar listesi tanımlanmış ve simülasyon ortamına kaydedilmesi için komut olarak yazılmıştır. (Son satır). Ayrıca her bir n değerli örneklem oluşturulması da orneklem olarak tanımlanmıştır.

```
[10] 1 sn.
▶ n10_ortalamlar = []
n30_ortalamlar = []
n100_ortalamlar = []
n500_ortalamlar = []

# n = 10 için 1000 kez simülasyon
for i in range(1000):
    orneklem = np.random.choice(kitle, size=10, replace=False)
    n10_ortalamlar.append(np.mean(orneklem))

# n = 30 için 1000 kez simülasyon
for i in range(1000):
    orneklem = np.random.choice(kitle, size=30, replace=False)
    n30_ortalamlar.append(np.mean(orneklem))

# n = 100 için 1000 kez simülasyon
for i in range(1000):
    orneklem = np.random.choice(kitle, size=100, replace=False)
    n100_ortalamlar.append(np.mean(orneklem))

# n = 500 için 1000 kez simülasyon
for i in range(1000):
    orneklem = np.random.choice(kitle, size=500, replace=False)
    n500_ortalamlar.append(np.mean(orneklem))
```

n=10 , n=30, n=100 ve n=500 yani her bir n değeri için 1000 tane örneklem elde edilmiştir. Yine her bir n değeri for döngüsünde ayrı ayrı ortalamaları yukarıda tanımlanan değerlerle hesaplanmıştır.

[11]
✓ 0
sn.

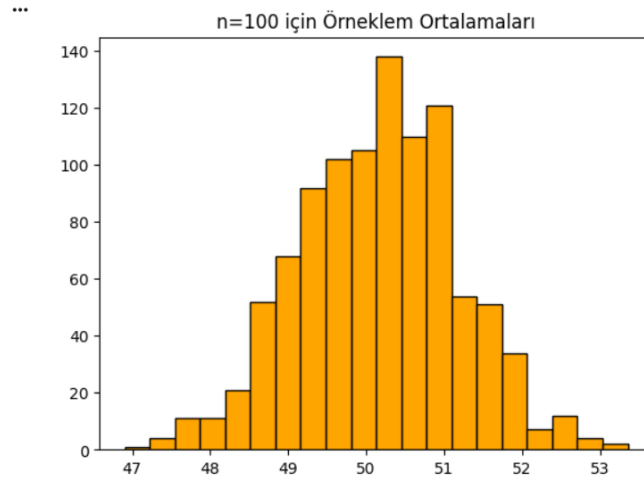
```
print("n=10 Ortalama:", np.mean(n10_ortalamalar), "Varyans:", np.var(n10_ortalamalar))  
print("n=30 Ortalama:", np.mean(n30_ortalamalar), "Varyans:", np.var(n30_ortalamalar))  
print("n=100 Ortalama:", np.mean(n100_ortalamalar), "Varyans:", np.var(n100_ortalamalar))  
print("n=500 Ortalama:", np.mean(n500_ortalamalar), "Varyans:", np.var(n500_ortalamalar))
```

✓

```
*** n=10 Ortalama: 50.26969257652085 Varyans: 9.339936507640255  
n=30 Ortalama: 50.2405730891854 Varyans: 3.1359442576569463  
n=100 Ortalama: 50.168550134730694 Varyans: 1.0394235075110465  
n=500 Ortalama: 50.14497285606126 Varyans: 0.17316968673601904
```

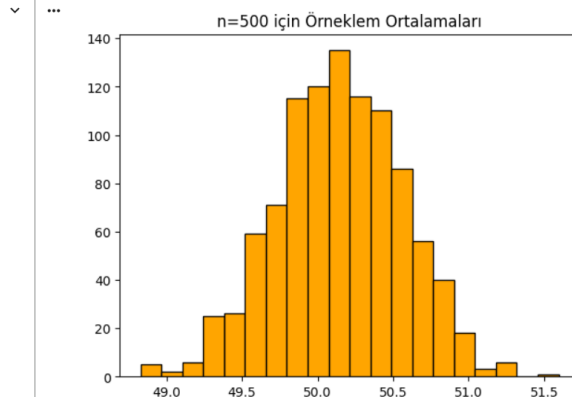
Her n değeri için hesaplanan ortalamalar ve varyanslar print fonksiyonuyla yazdırılmıştır.

```
# --- GRAFİK (Histogram) ---  
plt.hist(n100_ortalamalar, bins=20, color="orange", edgecolor="black")  
plt.title("n=100 için Örneklem Ortalamaları")  
plt.show()
```



[13]
✓ 0
sn.

```
# --- GRAFİK (Histogram) ---  
plt.hist(n500_ortalamalar, bins=20, color="orange", edgecolor="black")  
plt.title("n=500 için Örneklem Ortalamaları")  
plt.show()
```



n değerlerinin örneklem parametrelerinin dağılımlarının karşılaştırılması için önce n=100 daha sonra n=500 için histogram çizilmiştir. (plt.hist komutu)

ÖDEV 2

[14]
✓ 0
sn.

```
yeni_mu = 20  
yeni_sigma = 15  
N = 10000
```

Verilen kitle bilgileri değerleriyle beraber tanımlanmıştır.

[15]
✓ 1
sn.

```
kitle2 = np.random.normal(yeni_mu, yeni_sigma, N)  
print("Kitle Ortalaması:", np.mean(kitle2))  
print("Kitle Varyansı:", np.var(kitle2))
```

✓

```
Kitle Ortalaması: 19.80617352559966  
Kitle Varyansı: 231.5281125385894
```

Kitle bilgisi kitle2 olarak kaydedilmiştir.

[16]
✓ 0
sn.

```
▶ T1_ort = []  
T2_ort = []  
T3_ort = []
```

Kitle ortalaması için 3 tahmin edici tanımlanmıştır.

[17]
✓

```
T1_var = []  
T2_var = []
```

Aynı şekilde kitle varyansı için de verilen 2 tahmin edici tanımlanmıştır.

[18]

```
▶ for i in range(1000):  
    orneklem = np.random.choice(kitle2, size=30, replace=False)  
  
    # --- ORTALAMA TAHMİN EDİCİLERİ HESAPLAMA ---  
    t1_o = np.mean(orneklem)  
    T1_ort.append(t1_o)  
  
    t2_o = (orneklem[0] + orneklem[1]) / 2  
    T2_ort.append(t2_o)  
  
    t3_o = (orneklem[0] - 2*orneklem[1] + 3*orneklem[2]) / 6  
    T3_ort.append(t3_o)  
  
    # --- VARYANS TAHMİN EDİCİLERİ HESAPLAMA ---  
    t1_v = np.var(orneklem, ddof=1)  
    T1_var.append(t1_v)  
  
    t2_v = 0.5 * (orneklem[0] - orneklem[1])**2  
    T2_var.append(t2_v)
```

Bu kod bloğunda önce kitle ortalaması daha sonra kitle varyansı için verilen tahmin ediciler denklemleri yazılmış ve tanımlanmış daha sonra hesaplanmıştır.

[20]
0
✓ sn.

```
ort_tahminler = [T1_ort, T2_ort, T3_ort]
ort_isimler = ["T1", "T2", "T3"]
mu_gercek = 20
var_gercek = 225
```

Kitle ortalamasının tahmin edicileri T1, T2, T3 olarak yazılmış. Kitle ortalamasının gerçek değeri ve varyansı da tekrar hatırlatılmak için çağırılmıştır.

[]

```
print(f"{'Tahmin Edici':<20} | {'Beklenen Değer':<15} | {'Varyans':<10} | {'HKO':<10}")
print("-" * 65)

for isim, veriler in zip(ort_isimler, ort_tahminler):
    beklenen_deger = np.mean(veriler)
    varyans = np.var(veriler)
    hko = varyans + (beklenen_deger - mu_gercek)**2

    print(f"{isim:<20} | {beklenen_deger:<15.4f} | {varyans:<10.4f} | {hko:<10.4f}")
```

✓

Tahmin Edici	Beklenen Değer	Varyans	HKO
T1	19.7835	7.6414	7.6882
T2	20.0463	114.0191	114.0213
T3	6.6685	87.7064	265.4356

Kitle ortalamasının 3 tahmin edicisi için beklenen değer, varyans ve hata kareler ortalaması formülleri girilerek hesaplanmıştır. Daha sonra print fonksiyonuyla sütun isimleri verilmiş ve aralarındaki boşluk mesafesi belirlenerek ve sütun adlarının altına "-" konularak tablolaştırılmıştır.

[21]
0
✓ sn.

```
var_tahminler = [T1_var, T2_var]
var_isimler = ["T1", "T2"]

print(f"{'Tahmin Edici':<20} | {'Beklenen Değer':<15} | {'Varyans':<10} | {'HKO':<10}")
print("-" * 65)

for isim, veriler in zip(var_isimler, var_tahminler):
    beklenen_deger = np.mean(veriler)
    varyans = np.var(veriler)
    hko = varyans + (beklenen_deger - var_gercek)**2

    print(f"{isim:<20} | {beklenen_deger:<15.4f} | {varyans:<10.4f} | {hko:<10.4f}")
```

✓

Tahmin Edici	Beklenen Değer	Varyans	HKO
T1	231.1453	3557.4018	3595.1660
T2	233.8325	111930.4775	112008.4907

Bir önceki işlemin aynısı kitle varyansı tahmin edicileri için yapılmıştır.

[22]
0
✓ sn.

```
var_T1_ort = np.var(T1_ort)
var_T2_ort = np.var(T2_ort)

etkinlik_T2 = var_T1_ort / var_T2_ort

print(f"T2'nin T1'e göre etkinliği: {etkinlik_T2:.4f}")
```

✓

... T2'nin T1'e göre etkinliği: 0.0645

[23]
0
✓ sn.

```
var_T1_v = np.var(T1_var)
var_T2_v = np.var(T2_var)

etkinlik_var = var_T1_v / var_T2_v
print(f"T2'nin T1'e göre etkinliği: {etkinlik_var:.4f}")
```

✓

... T2'nin T1'e göre etkinliği: 0.0318

Varyans tahmin edicileri için etkinlik (T1'e göre karşılaştırma) hesaplanmıştır.

ÖDEV 3

[24]
0
✓ sn.

```
teta_gercek= 10
n = 30
tekrar_sayisi = 10000

teta_tahminleri = []

for i in range(tekrar_sayisi):

    orneklem = np.random.uniform(0, teta_gercek, size=n)

    tahmin = np.max(orneklem)

    teta_tahminleri.append(tahmin)
```

10.000 adet tahmini saklamak için boş liste oluşturulmuştur. U(0,10) dağılımından n=30 büyüklüğünde örneklem üretilip daha sonra her örneklem için en büyük değer hesaplanmıştır. Bu hesaplanan tahmin değerleri boş listeye eklenmiştir.

[25]
0
✓ sn.

```
ortalama_tahmin = np.mean(teta_tahminleri)
varyans_tahmin = np.var(teta_tahminleri)

print("Tahminlerin Ortalaması (E[teta_sapka]):", ortalama_tahmin)
print("Tahminlerin Varyansı:", varyans_tahmin)
```

✓

```
... Tahminlerin Ortalaması (E[teta_sapka]): 9.67662317164251
Tahminlerin Varyansı: 0.09638892970138313
```

Tahminlerin ortalama ve varyansları hesaplanmıştır.

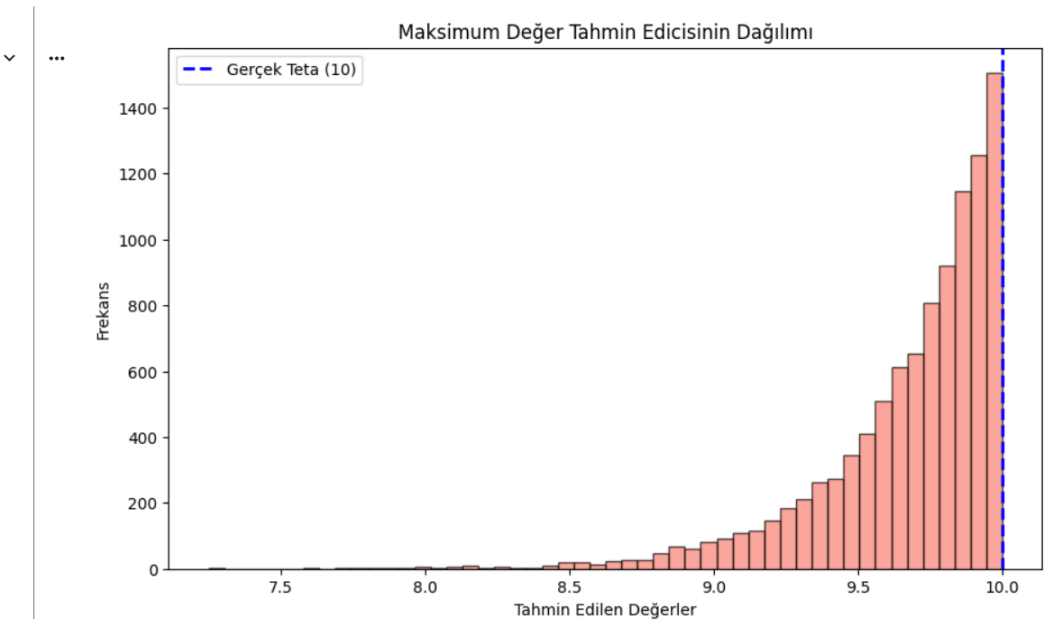
[26]
0
✓ sn.

```
plt.figure(figsize=(10, 6))

plt.hist(teta_tahminleri, bins=50, color='salmon', edgecolor='black', alpha=0.7)

plt.axvline(10, color='blue', linestyle='dashed', linewidth=2, label='Gerçek Teta (10)')

plt.title('Maksimum Değer Tahmin Edicisinin Dağılımı')
plt.xlabel('Tahmin Edilen Değerler')
plt.ylabel('Frekans')
plt.legend()
plt.show()
```



Tahmin edilen değerlerin dağılımını görebilmek için histogram grafiği çizdirilmiştir.

Kaynakça:

- Python Cookbook: Recipes for Mastering Python 3, David M. Beazley, Brian Jones
<https://elhacker.info/manuales/OReilly%204%20GB%20Collection/O'Reilly%20-%20Python%20Cookbook.pdf>
- <https://berkeyapa.medium.com/pythonda-%C3%B6rneklem-teorisi-uygulama-a02cb6e51e17> , Berke Yapa
- <https://levelup.gitconnected.com/visualizing-data-distributions-in-python-histograms-and-density-plots-f3f0d605efd3> , Ranjeet Tiwari
- <https://docs.python.org/3/> , Python Documentation
- https://numpy.org/doc/stable/user/absolute_beginners.html , NumPy
- İstatistiksel Yöntemlere Giriş, Dr. Haydar Demirhan, Dr. Canan Hamurkaroğlu